Lightweight Local AI Gateway & Inference Engine (No-Docker)

An optimized, zero-overhead deployment pipeline for running local Large Language Models (LLMs) on resource-constrained host environments. Built to bypass container virtualization overhead, optimize storage bandwidth across secondary partitions, and serve local network client requests.

📐 System Architecture & Hardware Requirements

This setup isolates LLM weights and database states from system OS partitions, maximizing I/O throughput and system stability during heavy VRAM allocation.

- **Primary Engine:** Ollama (C++ based Llama.cpp runner)
- **Serving Layer:** Open-WebUI (Python Native Virtual Environment)
- **Target Model:** Qwen2.5-Coder (Optimized 4-bit/8-bit Quantized Weights)
- **Recommended Specs:** 16GB RAM | 6GB Dedicated VRAM

⚙️ Key Technical Features

- **Zero-Docker Virtualization Overhead:** Native execution directly on host hardware for maximum VRAM efficiency and lower idle latency.
- **Storage-Aware Partition Routing:** Custom environment variable configuration to redirect weight stores (`OLLAMA_MODELS`) and state persistence (`DATA_DIR`) to secondary high-capacity media (`A:` drive).
- **Edge Network Accessibility:** Multi-device exposure across local subnets using socket binding (`0.0.0.0:8080`).
- **Automated Runtime Lifecycle:** Single-script execution pipeline (`launch.bat`) managing environment variables, background processes, and virtual environment activation.

🚀 Step-by-Step Deployment Guide

Step 1: Engine Initialization & Environment Routing

1. Install **Ollama** via the official installer.

2. Fully terminate the background instance via system tray before setting variables.
3. Configure persistent **User Environment Variables** to decouple heavy data from the primary `C:` OS drive:

Variable Name	Target Path	Engineering Rationale
`OLLAMA_MODELS`	`A:\ollama_storage`	Offloads quantized LLM weights off system OS partition
`DATA_DIR`	`A:\webui_data`	Isolates chat history, SQLite DB, and vector storage

4. Initialize physical storage targets:

```
mkdir A:\ollama_storage
mkdir A:\webui_data
```

Step 2: Native Environment Provisioning

Initialize an isolated Python virtual environment directly on the secondary drive:

```
:: Navigate to target drive partition
cd /d A:
mkdir local_ai_project
cd local_ai_project

:: Provision and activate isolated environment
python -m venv ai_env
ai_env\Scripts ctivate

:: Install web serving interface
pip install open-webui
```

Step 3: Deployment & Edge Network Binding

1. Restart the Ollama daemon.
2. Pull the targeted coding model:

```
ollama pull qwen2.5-coder
```

3. Launch the serving instance bound to all local network adapters:

```
open-webui serve --host 0.0.0.0 --port 8080
```

⚡ Automated Production Startup Script (`launch.bat`)

To streamline deployment without manual terminal entry, create a `launch.bat` script in your project root:

```
@echo off
TITLE Local AI Gateway Server
COLOR 0A

echo [1/3] Setting Storage Environment Variables...
SET OLLAMA_MODELS=A:\ollama_storage
SET DATA_DIR=A:\webui_data

echo [2/3] Starting Ollama Daemon...
start /b ollama serve

echo [3/3] Activating Virtual Environment & Launching Web Gateway...
call A:\local_ai_project i_env\Scripts ctivate
open-webui serve --host 0.0.0.0 --port 8080
```